

Orchestration with Airflow

Schedule, Monitor, and Manage Data Pipelines

■ AIRFLOW

■ LEARNING OUTCOME

By the end of this module you will write production Airflow DAGs, use all major operators, configure task dependencies, handle failures with retries and alerting, use sensors and XComs, and deploy Airflow with Docker Compose.

■ Skills You Will Gain

- Write Python DAGs with task dependencies and scheduling
- Use PythonOperator, BashOperator, and database operators
- Configure sensors: FileSensor, ExternalTaskSensor
- Pass data between tasks with XCom
- Set up retries, alerting, and SLA miss callbacks
- Deploy Airflow locally with Docker Compose

■ Airflow is on 70% of DE job descriptions. It's the glue that holds all data pipelines together — scheduling, monitoring, retrying, and alerting across your entire data platform.

SECTION 1

Airflow Architecture

Component	Role
DAG	Directed Acyclic Graph — Python file defining tasks and their dependencies. Lives in dags/
Operator	Template for a task type: PythonOperator, BashOperator, PostgresOperator, S3Sensor
Task	An instance of an operator with specific parameters — the actual unit of work
Scheduler	Reads DAG files, creates DAG Runs based on schedule, assigns tasks to executor
Executor	Runs tasks — SequentialExecutor (debug), LocalExecutor (one server), CeleryExecutor (dis
Web UI	Monitor DAG runs, view task logs, trigger manual runs, manage connections and variables
Metadata DB	PostgreSQL or MySQL database storing DAG, task, and run state — never use SQLite in pr
XCom	Cross-task communication — push/pull small values (IDs, row counts) between tasks

SECTION 2

Writing a Production DAG

```
from airflow import DAG from airflow.operators.python import PythonOperator from
airflow.operators.bash import BashOperator from airflow.providers.postgres.operators.postgres
import PostgresOperator from airflow.utils.dates import days_ago from datetime import datetime,
timedelta # Default args applied to ALL tasks in this DAG default_args = { 'owner': 'data_team',
'depends_on_past': False, # don't wait for previous run to succeed 'retries': 3, # retry 3 times on
failure 'retry_delay': timedelta(minutes=5), 'email': ['data-alerts@company.com'],
'email_on_failure': True, 'email_on_retry': False, 'execution_timeout': timedelta(hours=2), } with
DAG( dag_id='daily_sales_pipeline', default_args=default_args, schedule_interval='0 6 * * *', #
cron: 6 AM every day start_date=datetime(2024, 1, 1), catchup=False, # IMPORTANT: don't backfill
missed runs max_active_runs=1, # prevent parallel runs of same DAG tags=['sales', 'daily',
'production'], ) as dag: # Task 1: Extract new orders from source extract = PythonOperator(
task_id='extract_orders', python_callable=extract_orders_fn, op_kwargs={'date': '{{ ds }}'}, # {{
ds }} = execution date YYYY-MM-DD ) # Task 2: Run dbt transformations transform = BashOperator(
task_id='run_dbt_models', bash_command='cd /opt/dbt && dbt run --select sales_marts.* --target
prod', ) # Task 3: dbt test test = BashOperator( task_id='run_dbt_tests', bash_command='cd /opt/dbt
&& dbt test --select sales_marts.* --target prod', ) # Task 4: Refresh materialised view refresh =
PostgresOperator( task_id='refresh_dashboard_view', sql='REFRESH MATERIALIZED VIEW CONCURRENTLY
mv_daily_sales;', postgres_conn_id='prod_warehouse', ) # Define dependencies: extract -> transform
-> test -> refresh extract >> transform >> test >> refresh
```

SECTION 3

Sensors and XCom

```
from airflow.sensors.filesystem import FileSensor from airflow.sensors.external_task import
ExternalTaskSensor # FileSensor: wait for a file to appear before processing wait_for_file =
FileSensor( task_id='wait_for_partner_file', filepath='/data/landing/sales_{{ ds_nodash }}.csv',
poke_interval=300, # check every 5 minutes timeout=7200, # fail if not arrived within 2 hours
mode='reschedule', # release worker slot while waiting (not poke!) ) # ExternalTaskSensor: wait for
upstream DAG to complete wait_for_upstream = ExternalTaskSensor( task_id='wait_for_ingestion_dag',
external_dag_id='raw_data_ingestion', external_task_id='final_load',
execution_delta=timedelta(hours=1), # upstream runs 1h before this DAG timeout=3600,
```

```

mode='reschedule', ) # XCom: pass data between tasks
def extract_fn(**context):
    rows_extracted = run_extraction()
    # Push value to XCom – available to downstream tasks
    context['ti'].xcom_push(key='rows_extracted', value=rows_extracted)
def validate_fn(**context):
    # Pull value from upstream task's XCom
    rows = context['ti'].xcom_pull(task_ids='extract', key='rows_extracted')
    if rows == 0:
        raise ValueError('Extraction returned 0 rows – aborting pipeline')

```

SECTION 4

Key Airflow Concepts

Concept	Explanation
<code>{{ ds }}</code>	Jinja template: execution date in YYYY-MM-DD. Other macros: <code>{{ ds_nodash }}</code> , <code>{{ yesterday }}</code>
<code>catchup=False</code>	Don't create backfill runs for all dates since <code>start_date</code> . Always set <code>False</code> in production unless
<code>execution_date</code>	The logical date of the DAG run — NOT when it runs. A daily DAG scheduled for 6 AM Jan
Idempotent tasks	Tasks should produce same result if re-run with same date. Use date partitions, UPSERT op
Connection	Named credential stored in Airflow UI (Admin → Connections) — databases, S3, Slack, etc.
Variable	Key-value store in Airflow (Admin → Variables) — environment configs, toggle flags
SLA Miss	Alert when task exceeds expected duration — configure with <code>sla=timedelta(hours=1)</code> on task

■ PRACTICE Q & A

Q1. What does `catchup=False` do and why is it almost always set in production?

A: Prevents Airflow from creating DAG runs for all missed dates since `start_date`. Without it, a daily DAG with `start_date` 3 months ago would trigger 90 runs simultaneously and overwhelm your systems.

Q2. What is an XCom in Airflow and what are its limitations?

A: Cross-Communication: small values pushed/pulled between tasks via metadata database. Best for: row counts, file paths, status flags, IDs. NOT for: large DataFrames (it's stored in the metadata DB — max a few MB).

Q3. What is the difference between poke mode and reschedule mode in sensors?

A: Poke: sensor holds a worker slot while waiting — wastes resources. Reschedule: sensor releases the worker slot between checks — far more efficient for long waits. Always use reschedule mode in production.

Q4. Why should Airflow tasks be idempotent?

A: Tasks fail and get retried. A task that produces duplicate data when retried causes data quality issues. Idempotent tasks (using UPSERT, partition overwrite) produce the same result no matter how many times they run.

Q5. What is the `execution_date` in Airflow and why is it confusing?

A: The logical date the DAG run represents — NOT when the task physically executes. A daily DAG scheduled for 6 AM Jan 15 has `execution_date=Jan 14` (the period it covers). This trips up almost every beginner.

■ Airflow Cheat Sheet	
<code>DAG(dag_id, schedule_interval)</code>	Define pipeline DAG
<code>PythonOperator(task_id, callable)</code>	Run Python function
<code>BashOperator(task_id, bash_command)</code>	Run shell command
<code>t1 >> t2 >> t3</code>	Linear dependency chain
<code>[t1, t2] >> t3</code>	Multiple upstream tasks
<code>t1 >> [t2, t3]</code>	Parallel downstream tasks
<code>{{ ds }}</code>	Execution date YYYY-MM-DD

<code>catchup=False</code>	No backfill of missed runs
<code>retries=3, retry_delay=5min</code>	Retry on failure
<code>mode='reschedule'</code>	Efficient sensor mode
<code>xcom_push(key, value)</code>	Push value to XCom
<code>xcom_pull(task_ids, key)</code>	Pull value from XCom